

Ultimate JavaScript – Ejercicios de Programación –

Sección: objetos.

Ejercicio 1: Usuarios.

¿Cómo funciona?

Escribiremos una función constructora, para hacer nuestros usuarios lo haremos con los siguientes datos: nombre, apellido, fecha de nacimiento, dirección, edad, país de nacimiento, y si tiene una suscripción activa en la Academia Hola Mundo.

Y crearemos 3 usuarios, los cuales tendremos que imprimir en la consola.

Entradas.

Definiremos la función constructora para crear usuarios, y definiremos 3 usuarios usando la palabra reservada de new, con los siguientes datos:

Usuario 1

- Nombre: Chanchito
- Apellido: Feliz
- Fecha de Nacimiento: 10 de abril de 1992
- Dirección: Av. Siempre Viva 742
- Edad: 31 años
- País de Nacimiento: México
- Suscripción Activa: True

Usuario 2

- Nombre: Chanchito
- Apellido: Triste
- Fecha de Nacimiento: 25 de junio de 1985
- Dirección: Calle Luna 123
- Edad: 38 años
- País de Nacimiento: España
- Suscripción Activa: False

Usuario 3

- Nombre: Felipe
- Apellido: Schurmann
- Fecha de Nacimiento: 3 de septiembre de 2000
- Dirección: Boulevard del Sol 456
- Edad: 23 años
- País de Nacimiento: Argentina
- Suscripción Activa: true

Salidas.

Tendremos que ver los 3 usuarios en nuestra consola, con los datos correspondientes.

Código.

```
1 // Función constructora para crear usuarios
2 function Usuario(nombre, apellido, fechaNacimiento, direccion, edad, paisNacimiento, suscripcionActiva) {
3     this.nombre = nombre;
4     this.apellido = apellido;
5     this.fechaNacimiento = fechaNacimiento;
6     this.direccion = direccion;
7     this.edad = edad;
8     this.paisNacimiento = paisNacimiento;
9     this.suscripcionActiva = suscripcionActiva;
10 }
11
12 // Crear tres usuarios
13 const usuario1 = new Usuario("Chanchito", "Feliz", "10 de abril de 1992", "Av. Siempre Viva 742", 31, "México", true);
14 const usuario2 = new Usuario("Chanchito", "Triste", "25 de junio de 1985", "Calle Luna 123", 38, "España", false);
15 const usuario3 = new Usuario("Felipe", "Schurmann", "3 de septiembre de 2000", "Boulevard del Sol 456", 23, "Argentina", true);
16
17 // Imprimir los usuarios en la consola
18 console.log(usuario1);
19 console.log(usuario2);
20 console.log(usuario3);
21
```

Ejercicio 2: .

¿Cómo funciona?

Vamos a copiar la función constructora de nuestro ejercicio anterior, pero solamente vamos a crear un usuario. Y vamos primero a verificar si es que nuestro objeto tiene las propiedades: suscripción, dirección y altura.

Si este tiene el valor de la suscripción como true, lo pasaremos a false, y si es false, la modificaremos por true.

Mientras que si existen las propiedades: dirección y altura, vamos a borrar dichas propiedades del usuario que creamos.

Entradas.

Definiremos un usuario con los datos:

Usuario 1

- Nombre: Chanchito
- Apellido: Feliz
- Fecha de Nacimiento: 10 de abril de 1992
- Dirección: Av. Siempre Viva 742
- Edad: 31 años
- País de Nacimiento: México
- Suscripción Activa: true

Y realizaremos 3 validaciones para verificar que este objeto tenga las 3 propiedades, y realizaremos la acción correspondiente.

Salidas.

Imprimimos a nuestro usuario después de haber realizado las modificaciones. Deberás de ver que nuestro usuario tendrá en la propiedad **{ suscripcion }** con el valor de **{ false }** y no tiene que tener la propiedad **{ direccion }**.

Código.

```
1 // Definimos la función constructora
2 function Usuario(nombre, apellido, fechaNacimiento, direccion, edad, paisNacimiento, suscripcion) {
3     this.nombre = nombre;
4     this.apellido = apellido;
5     this.fechaNacimiento = fechaNacimiento;
6     this.direccion = direccion;
7     this.edad = edad;
8     this.paisNacimiento = paisNacimiento;
9     this.suscripcion = suscripcion;
10 }
11
12 // Creamos los usuarios
13 const usuario1 = new Usuario("Chanchito", "Feliz", "10 de abril de 1992", "Av. Siempre Viva 742", 31, "México", true);
14
15 if ("suscripcion" in usuario1) {
16     usuario1.suscripcion = !usuario1.suscripcion; // Alternar suscripción
17 }
18
19 if ("direccion" in usuario1) {
20     delete usuario1.direccion
21 }
22
23 if ("altura" in usuario1) {
24     delete usuario1.altura
25 }
26
27 // Mostramos los objetos modificados en la consola
28 console.log(usuario1);
29
```

Ejercicio 3: App para vender boletos de eventos.

¿Cómo funciona?

Vamos a generar una función constructora, para nuestra aplicación de eventos, pero en este caso venderemos boletos para un evento.

Este objeto deberá tener las siguientes propiedades:

- Nombre del evento,
- Duración,
- Boletos disponibles.

Y tendrá los siguiente métodos:

- Mostrar información : que nos deberá decir, la información del evento.
- Comprar boleto: que deberá reducir la cantidad de boletos disponibles según los boletos comprados. Solo permitiremos 5 boletos por compra. Si ya no hay boletos disponibles, tendremos que decirle al usuario que ya no hay boletos disponibles.
- Cancelar boleto: que deberá agregar la cantidad de boletos disponibles.

Entradas.

Definiremos la función constructora y definiremos un evento, con por lo menos 50 boletos disponibles.

Salidas.

Realizaremos las siguientes acciones:

- Mostraremos la información del evento.
- Compraremos 3 boletos.
- Mostraremos la información del evento.
- Intentaremos comprar 6 boletos.
- Cancelaremos 2 boletos.
- Mostraremos la información del evento.

Al final de todas estas acciones, el evento solo deberá tener 49 boletos.

Código.

```
1 // Función constructora para el evento
2 function Evento(nombre, duracion, boletosDisponibles) {
3     this.nombre = nombre;
4     this.duracion = duracion;
5     this.boletosDisponibles = boletosDisponibles;
6
7     // Método para mostrar la información del evento
8     this.mostrarInformacion = function () {
9         console.log(`Evento: ${this.nombre}`);
10        console.log(`Duración: ${this.duracion} horas`);
11        console.log(`Boletos disponibles: ${this.boletosDisponibles}`);
12    };
13
14    // Método para comprar boletos
15    this.comprarBoleto = function (cantidad) {
16        if (cantidad > 5) {
17            console.log("Solo puedes comprar hasta 5 boletos por compra.");
18            return;
19        }
20
21        if (this.boletosDisponibles ≥ cantidad) {
22            this.boletosDisponibles -= cantidad;
23            console.log(`Has comprado ${cantidad} boleto(s).`);
24        } else {
25            console.log("Lo sentimos, ya no hay suficientes boletos disponibles.");
26        }
27    };
28
29    // Método para cancelar boletos (devolverlos)
30    this.cancelarBoleto = function (cantidad) {
31        this.boletosDisponibles += cantidad;
32        console.log(`Has cancelado ${cantidad} boleto(s).`);
33    };
34 }
35
36 // Crear un evento
37 const concierto = new Evento("Concierto de Paul McCartney", 3, 50);
38
39 // Mostrar información inicial del evento
40 concierto.mostrarInformacion();
41
42 // Comprar boletos
43 concierto.comprarBoleto(3);
44 concierto.mostrarInformacion();
45
46 // Intentar comprar más de 5 boletos
47 concierto.comprarBoleto(6);
48
49 // Cancelar boletos
50 concierto.cancelarBoleto(2);
51 concierto.mostrarInformacion();
52
```

Ejercicio 4: Propiedad privada para app de boletos.

¿Cómo funciona?

Para este ejercicio retomaremos lo que hicimos en el anterior ejercicio, ya que haremos una modificación.

Para nuestro objeto, tenemos un problema, y es que la propiedad de boletos disponibles puede ser accedida desde fuera del objeto. Esto es un error, no deberíamos dejar que la cantidad sea modificada de esta manera, solo deberíamos permitir que los boletos se vendan o se devuelvan por medio de nuestros métodos, por lo que vamos a convertir a esta propiedad como privada.

Entradas.

Definiremos la función constructora y definiremos un evento, con por lo menos 50 boletos disponibles. Pero la propiedad de boletos la definiremos como propiedad privada.

Salidas.

Realizaremos las siguientes acciones:

- Mostraremos la información del evento.
- Compraremos 3 boletos.
- Mostraremos la información del evento.
- Intentaremos cambiar el valor de la propiedad de boletos a 100.
- Cancelaremos un boleto.
- Mostraremos la información del evento.

Al final de todas estas acciones, el evento solo deberá tener 48 boletos.

Código.

```
1 function Evento(nombre, duracion, boletosDisponibles) {
2   // Variable privada usando una closure
3   let _boletosDisponibles = boletosDisponibles;
4
5   // Método para mostrar la información del evento
6   this.mostrarInformacion = function () {
7     console.log(`Evento: ${nombre}`);
8     console.log(`Duración: ${duracion} horas`);
9     console.log(`Boletos disponibles: ${_boletosDisponibles}`);
10  };
11
12  // Método para comprar boletos
13  this.comprarBoleto = function (cantidad) {
14    if (cantidad > 5) {
15      console.log("Solo puedes comprar hasta 5 boletos por compra.");
16      return;
17    }
18
19    if (_boletosDisponibles ≥ cantidad) {
20      _boletosDisponibles -= cantidad;
21      console.log(`Has comprado ${cantidad} boleto(s).`);
22    } else {
23      console.log("Lo sentimos, ya no hay suficientes boletos disponibles.");
24    }
25  };
26
27  // Método para cancelar boletos (devolverlos)
28  this.cancelarBoleto = function (cantidad) {
29    _boletosDisponibles += cantidad;
30    console.log(`Has cancelado ${cantidad} boleto(s).`);
31  };
32 }
33
34 // Crear un evento
35 const concierto = new Evento("Concierto de Paul McCartney", 3, 50);
36
37 // Mostrar información inicial del evento
38 concierto.mostrarInformacion();
39
40 // Comprar boletos
41 concierto.comprarBoleto(3);
42 concierto.mostrarInformacion();
43
44 // Intentar modificar boletos directamente(esto NO es posible porque es privado)
45 concierto._boletosDisponibles = 100; // No tendrá efecto en la propiedad.
46
47 // Cancelar boletos
48 concierto.cancelarBoleto(1);
49 concierto.mostrarInformacion();
50
```


Ejercicio 5: Estandarizar biblioteca.

¿Cómo funciona?

Estamos actualizando el sistema de una biblioteca, tendremos un listado de objetos, cada uno de estos será un libro, lo que tendremos es estandarizar las propiedades que tiene cada uno de estos objetos. Todos los objetos tendrán que tener las siguientes propiedades: título, autor, año de publicación, categoría, número de páginas.

Así que deberás recorrer este array, verificar que el objeto tenga dicha propiedad y si no tiene esa propiedad, tendrás que agregarla añadiendo el valor de null para cada uno de ellos.

Entradas.

Tendremos que usar un array de nuestros libros, estos serán los valores para cada uno:

Libro 1

- Título: Libro A
- Autor: Autor A
- Categoría: Ficción
-

Libro 2

- Título: Libro B
- Autor: Autor B
- Año de Publicación: 2021
- Número de Páginas: 300

Libro 3

- Título: Libro C
- Categoría: Historia
- Número de Páginas: 150

También definiremos una función para estandarizar las propiedades de nuestros libros. Y la llamaremos pasando cada uno de los libros.

Salidas.

Imprimiremos nuestros libros, y tienen que tener 5 propiedades, donde si uno de los libros no los tenía con anterioridad, lo colocaremos el valor de null.

Código.

```
1 // Lista de libros (algunos con propiedades faltantes)
2 const libros = [
3   { titulo: "Libro A", autor: "Autor A", categoria: "Ficción" },
4   { titulo: "Libro B", autor: "Autor B", annoPublicacion: 2021, numeroDePaginas: 300 },
5   { titulo: "Libro C", categoria: "Historia", numeroDePaginas: 150 },
6 ];
7
8 // Función para estandarizar las propiedades de un libro
9 function estandarizarLibro(libro) {
10   const propiedadesRequeridas = [
11     "titulo",
12     "autor",
13     "annoPublicacion",
14     "categoria",
15     "numeroDePaginas"
16   ];
17
18   for (let i = 0; i < propiedadesRequeridas.length; i++) {
19     const propiedad = propiedadesRequeridas[i];
20     if (!(propiedad in libro)) {
21       libro[propiedad] = null; // Agrega la propiedad con valor null si no existe
22     }
23   }
24 }
25
26 // Recorremos la lista de libros y aplicamos la función
27 for (let i = 0; i < libros.length; i++) {
28   estandarizarLibro(libros[i]);
29 }
30
31 // Imprimimos el array actualizado
32 console.log(libros);
33
```

Ejercicio 6: Juego de cartas.

¿Cómo funciona?

Vamos a crear un juego de cartas, cada carta será un objeto, y esta tendrá diferentes propiedades y métodos.

Cada carta deberá tener:

- Nombre,
- Puntos de vida,
- Tipo,
- Uno o máximo 2 ataques, que deberán estar en un array. Cada ataque deberá tener los datos de puntos, que serán los puntos que resten al rival con el ataque, además del número de energías necesarias para realizar el ataque.
- Energía, que iniciará en 0
- Y tipos a los que la carta es débil.

Los métodos que debemos usar son:

- Atacar: este método recibirá, la carta que será el objetivo y el índice correspondiente de nuestro array de ataques.
 - Para este método verificaremos que el ataque exista en nuestro array de ataques, que nuestra carta tenga equipada energía necesaria para usar el ataque. Si no la tiene, le diremos a nuestro usuario que no tiene la suficiente energía.
 - Si tiene la energía, vamos a usar la carta que es objetivo y llamaremos al método para recibir el ataque.
 - Al final de este método debe restar la energía utilizada para el ataque.
- Recibir ataque: que deberá restar los puntos de ataque del ataque a los puntos de vida.
 - Si la carta ya no tiene puntos de vida, deberemos indicar que no se puede realizar el ataque y terminar la ejecución de este método.
 - Si tiene puntos de vida, tendremos que restar los puntos de ataque a los puntos de vida, y si los puntos de vida son menores o iguales a 0, tendremos que indicar que la carta ha sido derrotada.
- Añadir energía: que deberá sumar las unidades de energía que recibamos como parámetro.

Entradas.

Definiremos dos cartas, cada una con los siguientes datos:

Carta 1

- Nombre: Dragón de Fuego

- Puntos de Vida: 100
- Tipo: Fuego
- Ataques:
 - Lllamarada: 30 puntos de daño, requiere 2 de energía.
 - Golpe Ígneo: 50 puntos de daño, requiere 3 de energía.
- Debilidades: Agua

Carta 2

- Nombre: Serpiente Acuática
- Puntos de Vida: 120
- Tipo: Agua
- Ataques:
 - Tsunami: 40 puntos de daño, requiere 2 de energía
- Debilidades: Eléctrico

Salidas.

Vamos a ejecutar la siguiente secuencia para probar nuestra app:

- carta1.anadirEnergia(2);
- carta1.atacar(carta2, 0);
- carta2.anadirEnergia(3);
- carta2.atacar(carta1, 0);
- carta1.atacar(carta2, 1);
- carta1.anadirEnergia(3);
- carta1.atacar(carta2, 1)
- carta1.anadirEnergia(3);
- carta1.atacar(carta2, 1);
- carta1.anadirEnergia(2);
- carta1.atacar(carta2, 0);

Con esta secuencia, tendremos que ver los siguientes resultados:

Dragón de Fuego ha ganado 2 de energía. Energía actual: 2
Dragón de Fuego ataca con Lllamarada causando 30 de daño.
Serpiente Acuática ha recibido 30 de daño. Vida restante: 90

Serpiente Acuática ha ganado 3 de energía. Energía actual: 3
Serpiente Acuática ataca con Tsunami causando 40 de daño.
Dragón de Fuego ha recibido 40 de daño. Vida restante: 60

Dragón de Fuego no tiene suficiente energía para atacar.

Dragón de Fuego ha ganado 3 de energía. Energía actual: 3
Dragón de Fuego ataca con Golpe Ígneo, causando 50 de daño.
Serpiente Acuática ha recibido 50 de daño. Vida restante: 40

Dragón de Fuego ha ganado 3 de energía. Energía actual: 3
Dragón de Fuego ataca con Golpe Ígneo, causando 50 de daño.
Serpiente Acuática ha recibido 50 de daño. Vida restante: -10
Serpiente Acuática ha sido derrotado.

Dragón de Fuego ha ganado 2 de energía. Energía actual: 2
Dragón de Fuego ataca con Llamarada causando 30 de daño.
Esta carta ha sido derrotada, no tiene puntos de vida, el ataque no se realizó.

Código.

```

1 function Carta(nombre, puntosVida, tipo, ataques, debilidades) {
2     this.nombre = nombre;
3     this.puntosVida = puntosVida;
4     this.tipo = tipo;
5     this.ataques = ataques;
6     this.energia = 0;
7     this.debilidades = debilidades;
8
9     this.atacar = function (objetivo, ataqueIndex) {
10         if (ataqueIndex ≥ this.ataques.length) {
11             console.log(`${this.nombre} no tiene ese ataque.`);
12             return;
13         }
14
15         let ataque = this.ataques[ataqueIndex];
16         if (this.energia ≥ ataque.energiaNecesaria) {
17             console.log(`${this.nombre} ataca con ${ataque.nombre} causando ${ataque.puntos} de daño.`);
18             objetivo.recibirAtaque(ataque.puntos);
19             this.energia -= ataque.energiaNecesaria;
20         } else {
21             console.log(`${this.nombre} no tiene suficiente energía para atacar.`);
22         }
23     };
24
25     this.recibirAtaque = function (puntosAtaque) {
26         if (this.puntosVida ≤ 0) {
27             return console.log("Esta carta ha sido derrotada, no tiene puntos de vida, el ataque no se realizó");
28         }
29         this.puntosVida -= puntosAtaque;
30         console.log(`${this.nombre} ha recibido ${puntosAtaque} de daño. Vida restante: ${this.puntosVida}`);
31         if (this.puntosVida ≤ 0) {
32             console.log(`${this.nombre} ha sido derrotado.`);
33         }
34     };
35
36     this.anadirEnergia = function (energia) {
37         this.energia += energia;
38         console.log(`${this.nombre} ha ganado ${energia} de energía. Energía actual: ${this.energia}`);
39     };
40 }
41
42 // Creación de cartas
43 const carta1 = new Carta("Dragón de Fuego", 100, "Fuego", [
44     { nombre: "Llamarada", puntos: 30, energiaNecesaria: 2 },
45     { nombre: "Golpe Ígneo", puntos: 50, energiaNecesaria: 3 }
46 ], ["Agua"]);
47
48 const carta2 = new Carta("Serpiente Acuática", 120, "Agua", [
49     { nombre: "Tsunami", puntos: 40, energiaNecesaria: 2 }
50 ], ["Eléctrico"]);
51
52 // Simulación de juego
53 carta1.anadirEnergia(2);
54 carta1.atacar(carta2, 0);
55 carta2.anadirEnergia(3);
56 carta2.atacar(carta1, 0);
57 carta1.atacar(carta2, 1);
58 carta1.anadirEnergia(3);
59 carta1.atacar(carta2, 1);
60 carta1.anadirEnergia(3);
61 carta1.atacar(carta2, 1);
62 carta1.anadirEnergia(2);
63 carta1.atacar(carta2, 0);
64

```